Lighting, Shadows, Occlusion and Vegetation in a Deferred City Renderer

Internal research report for the macOS / Apple Metal port · November 2026
Companion code: `../metal/` (Metal port), `../reference_hlsl/` (recovered HLSL), `../tools/` (extraction).

> **Abstract.** We recovered the complete shader library of a Direct3D 9 city-building renderer from its retail data packages: 314 vertex and 512 pixel *shader fragments* in four quality tiers, plus 307 material definitions that list which fragments make up each shader. The renderer is a hybrid. A thin G-buffer (packed depth, normal, gloss) feeds screen-space passes for shadows, ambient occlusion, local lights and fog. Most materials are then shaded forward, reading those buffers back. Indirect light uses no dynamic GI. It combines a pre-integrated *Perez sky look-up table*, CPU-projected spherical harmonics, and *baked 3-D directional-occlusion volumes* that also emit a single tinted bounce. Vegetation runs on a three-tier impostor system: meshes, an 8-view normal+depth atlas, and GPU-culled procedural forest cards. We describe each technique as implemented, give the maths, and present a Metal port of the parts that matter for the macOS version. All six Metal source files compile with the macOS Metal compiler.

1. Recovering the shader library
2. Fragment-based shader composition
3. Frame structure
4. Lighting model and sky-based GI
5. Shadows
6. Directional AO volumes and bounce light
7. Vegetation
8. Reflections and water
9. Atmospherics
10. Quality tiers
11. Porting to Metal
12. Limitations and future work

1 Recovering the shader library

The game data ships as DBPF v3 packages. The header holds an index count at `0x24`, the index size at `0x2C` and a 64-bit index offset at `0x40`. The index begins with a flags word that marks which of *type / group / instance-high* are constant, followed by 28-byte entries. Payloads are RefPack/QFS-compressed (`0x10FB` signature). `tools/dbpf.py` implements the index and the decompressor.

All shader data sits under resource type `0x0469A3F7` in the application package, in four groups with instances 0–3 (one per quality tier):

Group	Size (tier 3)	Content
`0x40212002`	1.40 MB	Fragment library: big-endian; `u32 3`, `u32 1024`, then 1023 VS slots, `u32 1024`, 1024 PS slots (slot 0 empty). Each slot is a header, a main-code string, a declaration string, parameters (decl + 12-byte register metadata), PS-only sampler names, and a name.
`0x40212004`	6.3 MB	Material table (307 entries: supported VS/PS versions, flags, name, VS list, PS list of 26-byte *conditional fragment references*), followed by 1 702 precompiled DX9 SM3 blobs (fxc 9.29.952.3111, with `DBUG` chunks).
`0x40212005`	33 KB	Render-pass graph: pass names, render targets and debug views (§3).
`0x40212000`	22 KB	Small header/lookup table (not needed for reconstruction).

A material reference stores its fragment index in the last two bytes. Vertex indices are 1-based, while pixel indices use the raw slot (slot 0 is empty). We confirmed this mapping on several materials where the fragment sequence must be coherent. For example, `impostorTree` resolves to `unpackNormalTangent → impostorOffscreenInstanceData → impostorTreeVS`, which supplies the `angle` variable the next fragment needs. `tools/extract_fragments.py` writes every fragment as an `.hlsl` file and reassembles every material into one listing per tier.

The executable also holds the shader builder's *prelude* (`engine_notes/shader_builder_prelude.txt`): the `cVertIn/cVertCurrent/cVertOut` and `cFragIn/cFragCurrent/cFragOut` structures, and helper shims (`make_float3x3`, `greater_equal`, `toInt`) emitted for *both* an HLSL back-end and a GLSL back-end (`gl_FragData`, `texture2DLod`). In other words, the engine already ran its fragments through an OpenGL path, which is useful precedent for a Metal back-end.

2 Fragment-based shader composition

Shaders are not written whole. Each fragment is a short code snippet that reads and writes a shared `Current` structure, initialised from `In` and finally copied to `Out`. Interpolators are named symbolically (`Current.texcoord<t2>`) and samplers likewise (`Sampler<s1>`). The builder assigns physical registers. Each fragment declares its uniforms with register metadata, so the builder can pack constants without collisions. A material lists candidate fragments, and a 24-byte condition mask on each entry (vertex components present, instancing, shadow/deferred pass, data-view mode) decides which ones are spliced in. This one mechanism yields every permutation: forward vs deferred, instanced vs batched, skinned vs static.

```
// material "NonImpostorTree" (tier 3)
VS: whiteVertexColor, copyUV01Tree, worldSpaceNormalTangent, setupSHTangent,
shLightingDiffSpec9, toClipSpace
PS: deferredDiffuseDisabled, sampleTreeTextures, setupSHTangent, scLightingApplyNMap
```

Porting implication: on Metal the natural equivalent is a code generator that emits MSL from the same fragment graph, with function constants for the condition masks. The hand-written files in `metal/` are the fixed, readable form of the most important permutations.

3 Frame structure

The pass graph (`engine_notes/render_pipeline_passes_and_targets.txt`) runs in this order:

`terrain / grass / paths` `batchedModels` `terrainShadows` `water · waterHD` `impostors · impostorShadows · cityImpostors` `lots` `shadows` `forest` `atmospherics` `dof` `motionBlur` `bloom` `tonemap` `clut` `fxaa` `temporalSSAA` `ui`

Main pass order. Render targets include `depth`, `normal`, `diffuse`, `lightScalar`, `shadowMapDepthRT/ColorRT`, `HDRColor` (+ half/quarter res), `waterRefraction`, `fftA0–4/fftB0–4` and `waterHeight/Normals/Choppy`. Config switches: `configCascadedShadows, configCloudShadows, configAoShadows, configAoTerrainOntoWorld, configVolumetricLight, configAtmospherics`.

The G-buffer is minimal and LDR-friendly. Depth is packed into RGB8 (`dot(rgb, (1, 1/255, 1/255²))`) with spec power in A. Normal sits in RGB8 with gloss in A. The two key intermediates are `lightScalar` (R = sun shadow, G = ambient occlusion, both screen-space) and a light-accumulation buffer for deferred point/spot/line lights (rgb diffuse, a = specular). Materials consume both in a forward pass. In effect this is a *light pre-pass* renderer that keeps forward-shaded materials.

4 Lighting model and sky-based GI

4.1 Perez sky look-up table

Sky radiance follows the Perez/Preetham model in CIE xyY:

$$Y(\theta,\gamma) = (1 + A{\cdot}e^{B/\cos\theta}) \cdot (1 + C{\cdot}e^{D\gamma} + E{\cdot}\cos^2\gamma) \cdot Y_z$$

The shipping path does not evaluate the exponentials. A 2-D table (sampler s11) stores the θ term along `c = ½(cosθ+1)` and the γ term along `g = √(½(1−cosγ))`, quantised to 8 bits and dequantised with `sunSkyRadianceScales`. Rows `s ∈ {0,⅓,⅔,1}` hold progressively *convolved* versions of the sky, which gives glossy and diffuse pre-filtering from one 2-D texture. Saturation and luminance are tuned around the white point and converted to linear RGB with a fixed XYZ→RGB matrix.

4.2 SceneLighting composite

Nearly every lit material ends in one function (`Common.h: SceneLighting`):

```
dir     = lerp(N, reflect(V,N), 0.75·gloss)          // one LUT tap
diff    = SkyConv(dir, 1−s)·(1−s)·AO ;  spec = SkyConv(dir,1−s)·s·AO
sunTerm = pow(saturate(N·L), 1+gloss) · sunColor · shadow
hilite  = BlinnPhongNorm(N,H,e) · Schlick(F0) · specStrength · N·L · sunColor · shadow
diff   += sunTerm + lightBuffer.rgb ;  spec += lightBuffer.a
color   = albedo · (diff + spec) + hilite
```

Specular is energy-normalised Blinn-Phong (`(e+2)/8`) with Schlick Fresnel, approximated as `exp2(−8.65617·L·H)`. A `ClipToHemisphere(n, n_g)` helper bends the lookup direction toward a ground plane so terrain-hugging objects don't pick up light from below.

4.3 Spherical harmonics

The low tier, the effects, and close-range trees use 9-coefficient SH evaluated per vertex. Bands 1–2 are convolved with the cosine lobe ($\hat{A}_1 = 2/3$, $\hat{A}_2 = 1/4$) for diffuse, and evaluated *unconvolved* along R for the glossy term. A 16-coefficient variant adds the cubic band per pixel for sharper reflections. The CPU projects the same Perez sky into SH, so both paths match.

5 Shadows

Sun shadows are rendered into an R-channel linear-depth map, $(z_{cam} - near)/(far - near)$. Alpha-tested casters and tree cards reconstruct per-texel depth. Receivers never sample the shadow map in their own shaders. Instead, a full-screen `globalShadowShader` rebuilds world position from the G-buffer (`camPos + farRay · depth`, with the ray interpolated without perspective correction) and writes the filtered result into `lightScalar.x`. The source defines eight filter styles:

Style	Method	Use
2	Single compare	fallback
3	Manual 2×2 bilinear PCF	generic SM2
4 / 5	ATI Fetch4: bilinear PCF / four Fetch4 ≈ 4×4 Gaussian	ATI hardware
6	Hardware PCF (`tex2Dproj`)	NVIDIA
7	Four hardware-PCF taps at ±0.5 texel, weight ¼	shipping: fragment `renderShadowOrthoN` uses style *N*, and `renderShadowOrtho7` is the only one any material references (`globalShadowShader`, `globalLightsShader`)

8	Receiver-plane depth bias (Isidoro); commented “doesn't work”	disabled

Every style ends with `shadow *= shadow` (“plump, smooth, and darken shadow (important!)”). Squaring narrows the penumbra of the small kernel and deepens contact shadows. On Metal, style 6/7 maps to `depth2d::sample_compare` and styles 4/5 map to `gather()` (swizzle `.wzxy` for D3D's `.agbr`).

Cloud shadows multiply three tiled cloud textures at 200/230/350 m altitude into the same buffer. Each layer is projected along the sun direction and scrolled at its own velocity, combined as `0.45 + 0.55·saturate(c1.g + c2.b − ½c0.r)`. **Forest canopy shadow** on terrain is a single tap of the forest mask offset 0.0025 UV toward the sun, darkening by 0.9: an inexpensive soft canopy shadow with no shadow-map cost.

6 Directional AO volumes and bounce light

The renderer does not use screen-space AO. Each building carries a small 3-D texture baked offline. Each voxel stores the mean occluder direction **d** (RGB) and the occluded solid angle *w* as a fraction of the sphere (A; 0.5 = hemisphere). At runtime the volume's box is rasterised. Each pixel reconstructs its volume UVW from depth, offsets it along the normal by `tuning.z` voxels to avoid self-occlusion, and evaluates the cosine-weighted visible fraction of a spherical cap:

```
c0 = 1 − 2w ;  s0 = √(1 − c0²) ;  t = saturate((N·(−d) + s0) / 2s0)
F  = lerp( saturate(−c0)², 1 − saturate(c0)², t )        // ProjSolidAngle
AO = 1 − (1 − F·k_s)^k_c                                  // artist curve
```

The same file ships `CombineOverlap`, which merges two directed caps by interpolating between w0+w1 and max(w0,w1) according to their angular overlap. `BendNormal` derives a bent normal. These are the tools used to bake and merge volumes. The *AndBounce* variant writes `0.7·AO·tint` into the light buffer. Light blocked by a coloured building is re-emitted in that building's colour, which is a cheap one-bounce GI term. **Terrain onto world** (`applyTerrainAOToWorld`) fakes occlusion from the ground: occluded angle = ½·(1 − saturate(Δz/6 m))², with the occluder direction pointing straight down.

7 Vegetation

7.1 Three tiers

Tier	Material	Technique
Near	`NonImpostorTree`	Mesh; diffuse·AO texture; normal map; SH9 per vertex + sky composite per pixel

Atlas	`impostorTree`	Each species is rendered from 8 view angles into an atlas: pre-multiplied colour, packed normal+depth, AO
Far / forest	`treeParticleStandard/ForestCamFacing`	Camera-facing cards that sample the atlas column chosen by facing direction; per-pixel lighting from the atlas normal

7.2 Impostor atlas

`impostorTreeVS` rotates the mesh by `transform[angle]`, moves it into camera space, normalises it to the $[-1,1]$ cell, then scales and offsets it into atlas cell *angle*. The PS clips to the cell and writes:

- **Colour** = diffuse·AO.rgb, alpha = `diffuse.a · saturate(100·(AO.a − leavesFallen) + 1)`. AO.a stores a per-leaf age, so one scalar animates foliage loss for seasons, pollution and dying trees.
- **Normal+depth** in RGB8: xy = normal.xy·½+½, z = ½ + sign(n.z)·½·depth. Unpacking rebuilds n.z = $\pm\sqrt{(1-x^2-y^2)}$ and recovers the depth. This gives cards per-pixel depth, which they use for depth output and shadow casting.

7.3 Procedural forests on the GPU

Forest cards come from a fixed grid of quads. The *vertex* shader decides whether each tree exists, using three textures:

```
density = ecoMap.y(uv)
size    = density − sampleIndex          // stratified threshold per card
angle   = |N · normalize(N.xy,0)|        // terrain steepness
draw    = size>0 && angle<0.5 && height ≥ seaLevel + 8.5 m
scale   = lerp(0.3, 0.9, size) + frac(random)·0.05
corner *= draw                           // culled quads collapse to a point
```

Because `sampleIndex` is stratified per card, raising the eco-map density switches cards on smoothly, and the surplus becomes tree size. Painting or bulldozing forest therefore costs only a texture update.

7.4 Card geometry

Cards are expanded in camera space. A *parallax fix* rescales the projected xy by `w / (w − camOffset.z · P[2].w)`, so a card offset in depth projects as if it sat at the tree's centre. A per-model z-bias (`−1e−5·w`) orders overlapping cards. Cards shrink vertically when viewed from above (`saturate(dot(−camDir, up)) + distanceFade`) and fade between 3000 and 3750 m. The wind function (`CalculateTreeWindAnimation`: travelling wave, `sin(√t)` easing, decaying envelope) is present but compiled out with `if(false)`.

7.5 Colour variation and lighting

Each of four tree types holds an HSV range (`terrainForestInfoPS[4]`), lerped by vertex colour and applied in YIQ space: a hue rotation of the IQ chroma plane, then a saturation scale on IQ and a value scale on all three channels. This produces seasonal and species variety from one atlas. The atlas is sampled with a manual mip level ($\frac{1}{2}\cdot\log_2$ of the footprint, clamped to 5) to stop neighbouring cells from bleeding into each other. Colour is then un-premultiplied, and the card is lit by `SceneLighting` with a fixed foliage material: gloss 0.08, exponent 3, strength 0.2. `treeBirth` grows new trees in two phases: the trunk first (30% of the time), then the crown widens from 10% bottom-up.

8 Reflections and water

The renderer has no planar or screen-space-traced reflections. Reflections come from three sources: sky LUT lookups along R for every glossy material and for water, a static cube map on vehicles (`lerp(c, cube(R), 0.15·specStrength)`), and water refraction of the HDR buffer.

HD water uses an FFT ocean (`initFFT → FFTX → FFTY → WaterHeight/Choppy/Normals`). The vertex shader displaces a terrain-style grid vertically and also horizontally (choppy), clamped never to rise above sea level. The normal pass builds a coarse normal from three height taps and re-orients two tiled detail normal maps onto it. The surface shader combines:

- a stylised Fresnel, $\text{clamp}(0.925 - (5\cos^2\theta)^{\frac{1}{4}}, 0.01, 1)$;
- sky reflection along reflect(−V,N), plus Phong sun glints;
- screen-space refraction offset by $0.03\cdot\sin(\pi N.xy)$, rejected (and re-sampled unrefracted) when the refracted pixel lies in front of the water;
- depth-based absorption toward a dark basin colour over 70 m of water;
- shoreline foam whose depth comes from a ring of 8 terrain-height taps. Foam fades with distance (2–10 km) and suppresses specular.

9 Atmospherics

A full-screen pass adds aerial perspective. Extinction integrates exponential height fog along the ray in closed form, $K(e^{-\alpha\, h_p} - e^{-\alpha\, h_v})$ with $K = B_0/(\alpha\cdot\cos\theta)$. It is applied as $\tau = e^{-d^2}$: the quadratic optical depth keeps the near field clear. Two layers come from the weather system (`mDynamicWeather`), one for geometry and one for the sky dome. In-scatter colour is the sky LUT along the view ray, with the horizon lifted near the sun. Commented alternatives include a “Crytek-style” volumetric density and an auto-scaling density.

10 Quality tiers

All four tiers share the same fragment names. Tier 0 (shader model 2 header `fffe0200`) differs from tier 3 in 1 VS and 59 PS fragments. It uses `half` precision in the sky/env code, drops the `lightScalar` read (no screen-space shadow or AO in the composite), skips impostor normal mapping, and skips the top-down card flattening on forests. Tiers 1–3 differ only slightly from one another.

11 Porting to Metal

Issue	Resolution in `metal/`
Row-vector `mul(v,M)` with column-packed constants	Upload the same bytes; use `v * M` (`float4x4`, and `float3x4` for HLSL `float4x3`). No CPU transposes.
Packed RGB8 depth / normal+depth	Kept bit-exact (`UnpackDepth`, `PackDepth24`, impostor pack). Later, move to `r32Float` / `rg16Snorm` targets on Apple GPUs (tile memory makes the G-buffer cheap).
Fetch4 / HW PCF	`gather()` and `depth2d::sample_compare` with a `compare_func::less_equal` sampler.
`VPOS` + half-pixel offset	Metal `[[position]]` is already at pixel centres; the D3D9 `+0.5` is dropped.
Fragment splicing & conditions	Recommended: generate MSL from the fragment graph; map condition masks to `[[function_constant]]`s.
Engine-injected macros (`FOAM_*`, `WATER_*`)	Not present in data; placeholder constants in `ReflectionsWater.metal` must be replaced with the engine values.
Light pre-pass round trip	On Apple silicon, use programmable blending / imageblocks so lightScalar and the light buffer never leave tile memory.

Ported entry points (all compile with `swift tools/validate_metal.swift metal`):

File	Functions
Common.h	Perez sky LUT, SceneLighting, SH9/SH16, Blinn-Phong-Schlick, solid-angle AO maths, YIQ HSV tweak, packing
Trees.metal	treeParticleStandardVS, treeParticleForestVS, treeParticleForestPS, treeParticleDeferredPS, treeParticleShadowPS, impostorTreeVS/PS, TreeBirth, ForestGroundShadow
Shadows.metal	renderShadowMapVS/PS, alpha-mask caster, ShadowOrtho/Proj (styles 2–7), globalShadowPS, cloudShadowPS
AmbientOcclusion.metal	aoVolumeVS/PS (AO + bounce MRT), applyTerrainAOToWorldPS
GlobalIllumination.metal	shLightingDiffSpec9VS, shLightingGlossy16PS, sceneLightingApplyPS, pointLightPS, globalLightsPS

ReflectionsWater.metal	hdWaterVS, hdWaterNormalPS, hdWaterPS, VehicleReflection
Atmospherics.metal	atmosphericsPS

12 Limitations and future work

- The 24-byte condition masks are recorded but not fully decoded. The assembled material listings include every candidate fragment, with its mask, rather than one resolved permutation.
- Constant values set by the CPU (SH coefficients, Perez coefficients, AO tuning, parallel lights) are not in the shader data. They need a capture from the running game.
- The precompiled DX9 blobs were not disassembled. Their HLSL sources are available, and the blobs serve as a ground truth for register allocation if the MSL generator needs it.
- `globalLightsPS` in the port reconstructs the per-light loop that the original splits across `parallelLight{N}*` fragments. Its fill-light term is our reading of the uniforms, not recovered code.
- Pixel-exact validation against D3D9 captures is the next step once a Metal host harness exists.